

Fleet Safety Intelligence V2

วิเคราะห์สถาปัตยกรรม Google Apps Script และ Google Sheets ทั้งระบบ ประเมินจุดแข็ง จุดเปราะบางเชิงเทคนิค พร้อมข้อเสนอแบบร่าง Playback Deep-Link สำหรับการตรวจสอบเหตุการณ์ และแผนพัฒนาต่อที่จัดลำดับความสำคัญแล้ว

จัดทำโดย ทีมวิเคราะห์ระบบ
วันที่ 8 ตุลาคม 2026

สารบัญ

บทที่ 1 บทสรุปผู้บริหาร	1
บทที่ 2 ภาพรวมระบบและวงจรการทำงาน	2
2.1 ระบบนี้แก้ปัญหาอะไร	2
2.2 วงจรการทำงานของระบบ	2
2.3 ไทม์ไลน์การพัฒนาและเหตุผลของแต่ละเวอร์ชัน	3
บทที่ 3 สถาปัตยกรรมเทคนิคแบบลึก	5
3.1 ชั้นข้อมูล: สัญญากับ Absolute API	5
3.2 ชั้นประมวลผล: กลไกที่ทำให้ GAS รองรับงานจริง	5
3.3 ชั้นจัดเก็บ: Google Sheets สืบตารางสามกลุ่ม	6
3.4 ชั้นนำเสนอ: สองช่องทางแจ้งเตือนกับหนึ่งแผนที่	6
3.5 ทะเบียนโมดูลซอฟต์แวร์	7
บทที่ 4 กฎธุรกิจและอัลกอริทึมสำคัญ	8
4.1 เกณฑ์จับความเร็วสองโหมด	8
4.2 Ray Casting และข้อจำกัดที่ควรรู้	9
4.3 ระบบคะแนน หักแต้ม และเกรด	9
4.4 ความเสี่ยง KPI และแผนโค้ชชิ่ง	9
บทที่ 5 ผลการตรวจสอบ: จุดแข็งของระบบ	10
บทที่ 6 จุดอ่อนและความเสี่ยงเชิงสถาปัตยกรรม	11
6.1 Google Sheets เป็นฐานข้อมูลเดียวของระบบ	11
6.2 การเขียนข้อมูลแบบล้างก่อนเขียน (Destructive Write)	11
6.3 ความล้มเหลวแบบเงียบ (Silent Failure)	12
6.4 ความปลอดภัยของ WebApp และการจัดการความลับ	12
6.5 ข้อจำกัดโควตาของแพลตฟอร์มที่ยังไม่มีตัวตรวจ	13
6.6 คุณภาพโค้ดและความเปราะบางของ Schema	13
บทที่ 7 Playback และการตรวจสอบเหตุการณ์ (Incident Investigation)	14
7.1 ทำไม Playback คือหัวใจของ Fleet Safety จริง	14

7.2 สถาปัตยกรรมการเชื่อมต่อกล้องปัจจุบัน (CMSV6 / MDVR)	15
7.3 ช่องว่างเชิงออกแบบและความเสี่ยงที่ต้องปิด	15
7.4 ข้อเสนอการออกแบบ: Playback Deep-Link ผ่านบริการนายหน้า	16
7.5 แผนทดสอบสามขั้นก่อนพัฒนาจริง	16
บทที่ 8 แผนพัฒนาต่อและลำดับความสำคัญ	17
8.1 งานค้างเดิมที่วางไว้แล้ว	17
8.2 งานเพิ่มเติมจากผลการตรวจสอบ	17
8.3 ตารางลำดับความสำคัญรวม	17
บทที่ 9 สรุปและข้อเสนอแนะเชิงปฏิบัติ	19

บทที่ 1 บทสรุปผู้บริหาร

รายงานฉบับนี้จัดทำขึ้นจากการอ่านและตรวจสอบเอกสาร Forensic Export ของโปรเจกต์ **Fleet Safety Intelligence V2** อย่างละเอียดทั้งสี่ส่วน ครอบคลุมประวัติการพัฒนา สถาปัตยกรรม ทะเบียนโค้ด โครงสร้างฐานข้อมูล กฎธุรกิจ อัลกอริทึม ประวัติข้อผิดพลาด การตัดสินใจทางเทคนิค และ Source Code ฉบับเต็มของโมดูลหลักทั้งหมด

วัตถุประสงค์ของรายงานคือการเปลี่ยนเอกสารสู่คืนสถานะ ให้กลายเป็นมุมมองเชิงที่ปรึกษาที่ตอบคำถามสามข้อในเวลาเดียวกัน ได้แก่ ระบบนี้ทำงานอย่างไร จุดแข็งและจุดเปราะบางอยู่ตรงไหน และควรลงทุนพลังงานในการพัฒนาต่อจากจุดใดก่อน

< 20 วินาที

เวลาดึงข้อมูล GPS ทั้งฟลีต
(จากเดิม 6 นาที)

18 โมดูล

ไฟล์ Google Apps Script
ในสถาปัตยกรรมปัจจุบัน

10 ตาราง

ฐานข้อมูล Google Sheets
ครบ 3 กลุ่มฟังก์ชัน

2 ช่องทาง

Telegram (เต็มรูปแบบ) +
LINE (ประหยัดโควตา)

โดยภาพรวม ระบบนี้เป็นตัวอย่างที่ดีมากของการสร้างระบบ Fleet Safety ระดับใช้งานจริงบนแพลตฟอร์ม Low-cost โดยไม่มีค่าเช่าเซิร์ฟเวอร์ การเลือก Google Apps Script ร่วมกับ Google Sheets ทำให้ทีมเล็ก สามารถส่งมอบความสามารถที่หลากหลายได้แก่ การดึงข้อมูล GPS แบบขนาน การวิเคราะห์ความเร็วแยกสองโหมด การตรวจจับเขตชุมชนด้วย Geofence ระบบให้คะแนนและตัดเกรดพนักงานขับรถ จนถึงแผนการโค้ชชิงอัตโนมัติ

ความเร็วของ Pipeline ที่ปรับจาก 6 นาทีเหลือไม่ถึง 20 วินาที แสดงให้เห็นว่าทีมเข้าใจข้อจำกัดของแพลตฟอร์ม และแก้ปัญหาอย่างตรงจุด

อย่างไรก็ตาม การตรวจสอบพบจุดเปราะบางเชิงสถาปัตยกรรมที่ควรเร่งจัดการภายในไตรมาสนี้ ได้แก่ การใช้ Google Sheets เป็นฐานข้อมูลเดียวทั้งระบบที่โตขึ้นทุกวันโดยไม่มีแผนเก็บถาวร กระบวนการเขียนข้อมูล

แบบล้าก่อนเขียนที่ไม่มีกลไกสำรองป้องกันการสูญเสียข้อมูลหากสคริปต์ขัดข้องกลางทาง การแจ้งเตือนความผิดพลาด ที่ยังเป็นแบบเงียบ (Silent Failure) และช่องว่างความปลอดภัยของ WebApp ที่เปิดให้อ่านข้อมูลรถได้จากลิงก์โดยตรง นอกจากนี้

ความต้องการเชิงธุรกิจที่สำคัญที่สุดคือ การดูวิดีโอย้อนหลัง (Playback) ณ

วินาทีที่ระบบแจ้งเตือน ซึ่งยังไม่ได้ถูกออกแบบลงในระบบปัจจุบัน รายงานฉบับนี้จึงอุทิศบทที่ 7 ให้กับการวิเคราะห์และเสนอแบบร่าง โซลูชัน Playback Deep-Link ที่ผูกกับเวลาเหตุการณ์โดยตรง

ข้อเสนอแนะเชิงปฏิบัติทั้งหมดถูกจัดลำดับเป็นแผนงานสามระดับ (PO ภายใน 2 สัปดาห์, P1 ภายในเดือน, P2 ภายในไตรมาส) ในบทที่ 8 โดยงานกลุ่ม PO ที่ควรเริ่มต้นที่คือการทดสอบ Playback API ของกล้อง MDVR การเปิดระบบสำรองข้อมูลรายวัน

และการติดตั้งช่องทางแจ้งเตือนเมื่อระบบทำงานผิดพลาด ซึ่งทั้งสามงานนี้ลงทุนเวลาไม่มากแต่ลดความเสี่ยงขององค์กรได้มากที่สุด

บทที่ 2 ภาพรวมระบบและวงจรการทำงาน

2.1 ระบบนี้แก้ปัญหาอะไร

Fleet Safety Intelligence V2

คือระบบติดตามและวิเคราะห์พฤติกรรมการขับรถเร็วเกินกำหนดของฟลีตรถบรรทุก โดยอาศัยข้อมูลจากอุปกรณ์ GPS ของผู้ให้บริการ Absolute จุดเริ่มต้นของโปรเจกต์ (รุ่น V1) มีข้อจำกัดสามข้อที่ทำให้ใช้งานได้จริงไม่เพียงพอ ประการแรกการดึงข้อมูลแบบทีละคัน (Sequential Fetch) กินเวลารวมถึง 6 นาทีต่อรอบ จนชนขีดจำกัดรันไทม์ของสคริปต์ ประการที่สองการแจ้งเตือนอาศัยระบบ Alarm ของผู้ให้บริการ GPS ล้วน ๆ ทำให้ปรับเกณฑ์และรูปแบบการแจ้งเตือนไม่ได้เอง ประการที่สามลิงก์แผนที่ในการแจ้งเตือนเป็น Google Maps จุดเดียว ที่ไม่เห็นบริบทของเส้นทางการขับเร็วทั้งเหตุการณ์

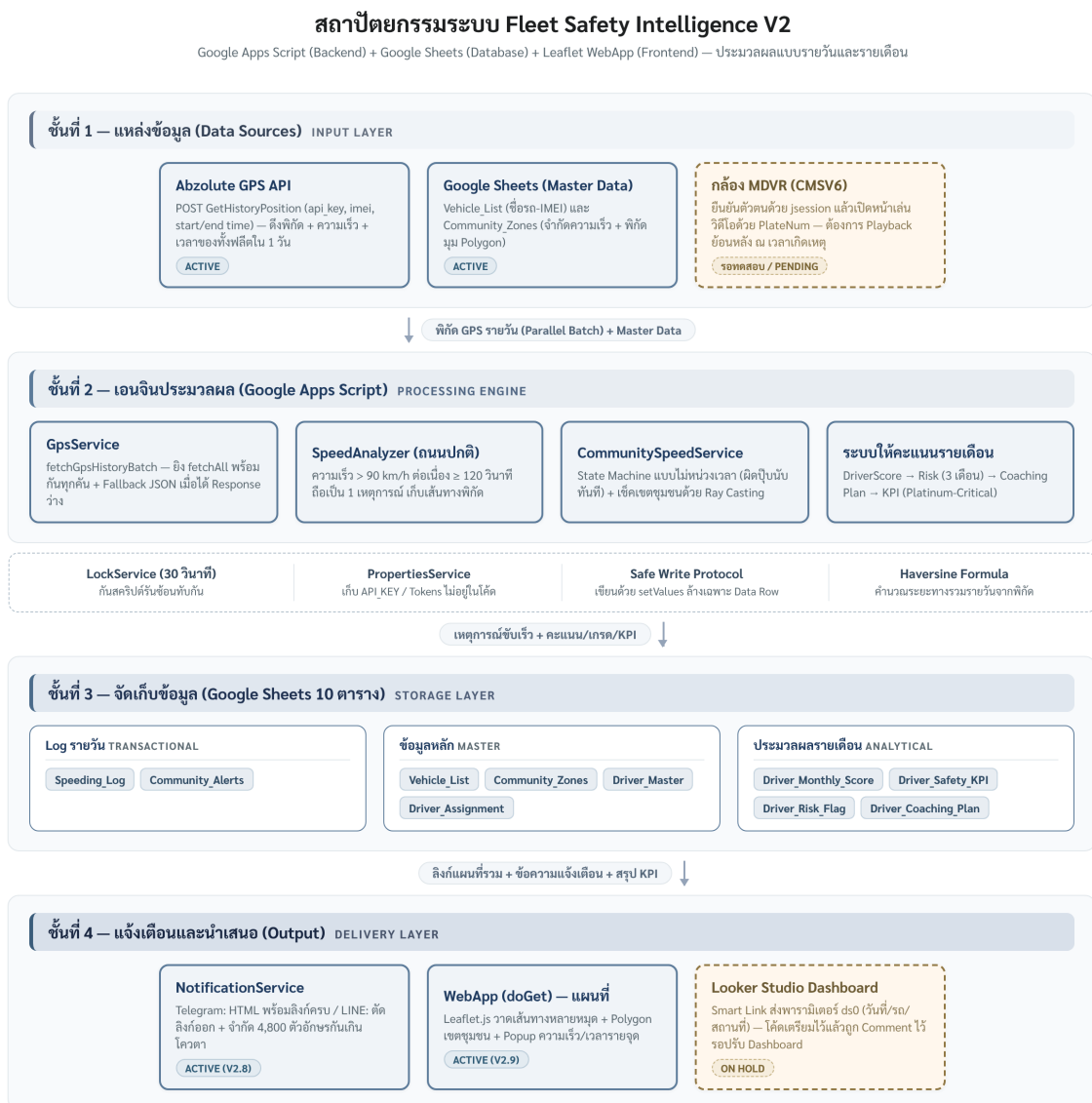
รุ่น V2 จึงถูกออกแบบใหม่ทั้งหมดบน Google Apps Script โดยวางเป้าหมายสามระดับ ระดับแรกคือความเร็ว ผ่านการดึงข้อมูลแบบขนานพร้อมกันทุกคัน (Parallel Batch Fetch) ระดับที่สองคือความแม่นยำของการนับเหตุการณ์ ผ่านเอนจินวิเคราะห์ความเร็วสองโหมดที่แยกพฤติกรรมบนถนนปกติกับเขตชุมชนออกจากกันอย่างเด็ดขาด และระดับที่สามคือความคุ้มค่าระยะยาว ผ่านการใช้โครงสร้างพื้นฐานของ Google Workspace ที่ไม่มีค่าเช่าเซิร์ฟเวอร์ และผูกเข้ากับเครื่องมือรายงานผลอย่าง Looker Studio ในอนาคต

2.2 วงจรการทำงานของระบบ

วงจรประจำวันเริ่มจากการอ่านรายชื่อรถจากตาราง Vehicle_List แล้วยิงคำขอไปยัง Absolute API พร้อมกันทุกคันเพื่อดึงพิกัด GPS ของวันที่ต้องการในช่วงเวลา 03:00 ถึง 22:00 (เวลา GMT+7) ข้อมูลที่ได้จะถูก บ้อนเข้าเอนจินวิเคราะห์สองตัวพร้อมกัน ตัวแรกคือ SpeedAnalyzer จับเหตุการณ์ขับเร็วบนถนนปกติที่เกิน 90 กิโลเมตร ต่อชั่วโมงต่อเนื่องอย่างน้อย 2 นาที ตัวที่สองคือ CommunitySpeedService ที่ตรวจว่าพิกัดขณะนั้นตกอยู่ใน Polygon เขตชุมชนหรือไม่แล้วนับความผิดปกติแบบไม่มีการหน่วงเวลา เหตุการณ์ทั้งหมดถูกบันทึกลงซีต พร้อมสร้างลิงก์แผนที่รวมต่อรถ แล้วสรุปส่งเข้า Telegram และ LINE โดยอัตโนมัติ

นอกจากวงจรรายวัน ระบบยังมี Pipeline รายเดือนแบบครบวงจร (End-to-End Monthly Pipeline) ที่รวบรวม Log ทั้งเดือนมาคำนวณคะแนนความปลอดภัยต่อรถ (DriverScoreService)

วิเคราะห์กลุ่มเสี่ยงจาก ประวัติ 3 เดือนย้อนหลัง (DriverRiskService)
ออกแผนการโค้ชซึ่งตามระดับความรุนแรง (CoachingService) และตัดเกรด KPI
รายบุคคลโดยเชื่อมกับตารางมอบหมายงานขับ (DriverKPIService) ผลลัพธ์ทั้งหมดเขียนลง
ชีตวิเคราะห์สีตารางและสรุปส่งเข้าช่องทางแจ้งเตือนในคลิกเดียวจากเมนูบน Google Sheets



ภาพที่ 1: สถาปัตยกรรมระบบ Fleet Safety Intelligence V2 แบ่งเป็น 4 ชั้น พร้อมสถานะงานที่รอเปิดใช้งาน

2.3 ไลน์การพัฒนาและเหตุผลของแต่ละเวอร์ชัน

เอกสาร Forensic บันทึกลำดับการพัฒนามีหลักฐานชัดเจน ตารางต่อไปนี้สรุปเวอร์ชันสำคัญสาเหตุที่ต้องเปลี่ยนแปลง และบทเรียนที่ได้จากแต่ละขั้น

จะเห็นได้ว่าการพัฒนาเกือบทั้งหมดถูกขับเคลื่อนด้วย ปัญหาจริงที่เกิดขึ้นในการใช้งาน ไม่ใช่การออกแบบล่วงหน้าจากทฤษฎี ซึ่งเป็นแนวทางที่เหมาะสมกับระบบที่สร้างบนแพลตฟอร์มที่มีข้อจำกัดหลายชั้นอย่าง Google Apps Script

ตารางที่ 1: ไทม์ไลน์การพัฒนา Fleet Safety Intelligence V2

เวอร์ชัน	เหตุการณ์	สิ่งที่เปลี่ยนแปลงและเหตุผล
V2.0	ย้ายบ้านสู่ Google Apps Script	ย้ายระบบแจ้งเตือนและวิเคราะห์มาบน GAS ดึงข้อมูลจาก Absolute API พร้อมเปลี่ยนการดึงข้อมูลเป็น Parallel Batch ลดเวลาจาก 6 นาทีเหลือเสี้ยววินาที
V2.3	แผนที่หลายจุด (เขตชุมชน)	WebApp รับพิกัดหลายจุด (pts) และวาด Polygon เขตชุมชนจากตาราง Community_Zones
V2.4-2.5	แผนที่รวมถนนปกติ	ขยาย WebApp รองรับเส้นทางขับเร็วบนถนนทั่วไป (ความเร็วเกิน 90 km/h)
V2.6-2.6.1	รวมลิงก์ต่อรถเดียว	รวบรวมทุก Session ของรถ 1 คันใน 1 วันให้อยู่ในลิงก์เดียว และเขียนสูตร =HYPERLINK เฉพาะแถวแรกของกลุ่มเพื่อลดความซ้ำซ้อน
V2.7	แก้ปัญหาลิมิต LINE	ข้อความแนบพิกัดยาวเกิน 5,000 ตัวอักษรจน LINE ยิง Error จึงสร้างฟังก์ชัน Downsample สุ่มพิกัดไม่เกิน 15 จุดและเพิ่ม Backward Compatibility ให้ WebApp
V2.8	ประหยัดโควตา LINE	LINE นับโควตาเป็นตัวอักษร จึงถอดแท็กลิงก์ HTML ออกจากข้อความ LINE และคงลิงก์เต็มไว้ใน Telegram
V2.9	State Machine เขตชุมชน	เปลี่ยนลอจิกเขตชุมชนเป็น State Machine แบบไม่มีการหน่วงเวลา ผ่อนแล้วเหยียบใหม่นับเป็น 2 ครั้งทันที พร้อมส่งความเร็วและเวลารายจุดเข้า Popup ของแผนที่

สถานะปัจจุบันของระบบคือการใช้โมดูลผสมระหว่าง V2.8 (ฝั่งการแจ้งเตือน) และ V2.9 (ฝั่งวิเคราะห์ความเร็ว และ WebApp) ซึ่งทำงานร่วมกันได้สมบูรณ์ เอกสารผู้ค้นยืนยันว่าไม่มี Known Bugs ค้างอยู่ ณ วันที่ทำ Forensic Export โดยงานที่ยังเปิดไม่ได้มีเพียงสามชิ้น ได้แก่ Looker Studio (โค้ดรอเปิด) AppSheet (รอทดสอบ Column Index) และการเชื่อมโยงวิดีโอย้อนหลังของกล้อง MDVR (รอพิสูจน์ URL จริงจากผู้ให้บริการ)

บทที่ 3 สถาปัตยกรรมเทคนิคแบบลึก

3.1 ชั้นข้อมูล: สัญญา กับ Absolute API

ระบบสื่อสารกับผู้ให้บริการ GPS ผ่านจุดปลายเดียวคือ service.php ด้วยวิธี POST มีเมธอดหลักสองตัว ตัวที่ใช้งานจริงคือ GetHistoryPosition รับ api_key, imei, start_time และ end_time เพื่อดึงประวัติภัยย้อนหลัง ทีละคัน ส่วน GetAlarmList ที่เคยใช้ดึงประวัติแจ้งเตือนจากระบบแม่ในรุ่น V2.0 ถูกเลิกใช้หลังจากที่ระบบเปลี่ยนไปคำนวณความเร็วเองทั้งหมด รูปแบบคำตอบเป็น JSON ที่มี error_code และอาเรีย data ประกอบด้วย latitude, longitude, speed และ position_timestamp โค้ดฝั่ง Parser ยังเตรียมรับชื่อฟิลด์แปรผันเช่น lat, lng, Speed และ time เพื่อกันผู้ให้บริการปรับเปลี่ยนโดยไม่แจ้ง

การระบุตัวตนรถใช้คู่คีย์สองชั้นคือชื่อย่อ (เช่น บธ-7049) และ IMEI 15 หลักของอุปกรณ์ โดยตาราง Vehicle_List ทำหน้าที่แปลงชื่อย่อเป็น IMEI ก่อนยิง API ข้อมูลวิเคราะห์ที่ใช้จริงมีเพียงสี่ฟิลด์ (พิกัด ความเร็ว เวลา) ฟิลด์อื่นเช่น Odometer หรือสถานะกุญแจจุดร้อนยังไม่ถูกใช้ ซึ่งเป็นข้อสังเกตสำคัญเพราะระยะทางรวมรายวันที่เพิ่มเข้ามาในช่วงท้ายจึงต้องคำนวณเองจากพิกัดด้วยสูตร Haversine แทนการอ่านเลขไมล์จากอุปกรณ์โดยตรง

3.2 ชั้นประมวลผล: กลไกที่ทำให้ GAS รองรับการจริง

หัวใจของชั้นประมวลผลคือ MainProcess ที่ทำหน้าที่ Orchestrator โดยเริ่มด้วยการครอง LockService เพื่อกันการรันซ้อนทับ (รอสูงสุด 30 วินาที) จากนั้นส่งคำขอตีพิมพ์ผ่าน UrlFetchApp.fetchAll ในครั้งเดียว เมื่อได้คำตอบว่างจาก API บางคัน ระบบจะสะสมรายการเหล่านั้นแล้วยิงซ้ำเป็นรอบ Retry โดยเปลี่ยน Content-Type เป็น application/json เทคนิคสองชั้นนี้คือสาเหตุที่ระบบเสถียรขึ้นมาก ทั้งเรื่องความเร็วและความทนทานต่อ พฤติกรรมแปลก ๆ ของผู้ให้บริการ API

เหนือข้อมูลดิบ ระบบมีกลไกประมวลผลเชิงพื้นที่และเชิงเวลาสามชุด ชุดแรกคือ Ray Casting ใน Utils.isPointInPolygon ที่ตรวจว่าพิกัดอยู่ใน Polygon เขตชุมชนหรือไม่ด้วยหลักนับจำนวนจุดตัดของเส้นแนวนอน ชุดที่สองคือ State Machine ของ CommunitySpeedService ที่เปิดสถานะเมื่อความเร็วเกินค่ากำหนดของโซน เก็บพิกัดสะสมและความเร็วสูงสุด แล้วปิดสถานะทันทีเมื่อผ่อนลงหรือออกนอกโซน ทำให้พฤติกรรมเร่ง-ผ่อน-เร่ง ถูกนับเป็นสองครั้งตามนโยบายความปลอดภัยเข้มงวด ชุดที่สามคือ

Downsample ที่ย่อจำนวนพิกัดจากหลายร้อยจุด ให้เหลือ 15 จุด (เขตชุมชน 20 จุด) โดยเก็บจุดแรกและจุดสุดท้ายไว้เสมอ เพื่อให้ URL แผนที่ไมยาวเกินขีดจำกัดของช่องทางแจ้งเตือน

3.3 ชั้นจัดเก็บ: Google Sheets สิบตารางสามกลุ่ม

ตารางที่ 2: โครงสร้างฐานข้อมูล Google Sheets ทั้ง 10 ตาราง

กลุ่ม	ชื่อตาราง	บทบาทและคอลัมน์สำคัญ
Transactional	Speeding_Log	Log เหตุการณ์ถนนปกติรายวัน: วันที่, ชื่อรถ, IMEI, เวลาเริ่ม-สิ้นสุด, ระยะเวลา, ความเร็วสูงสุด, ลิงก์แผนที่เดียว, ลิงก์แผนที่รวม
Transactional	Community_Alerts	Log เหตุการณ์เขตชุมชน: เพิ่มจุดชุมชน (FK), ความเร็วที่ใช้/ที่กำหนด, พิกัดจุดสำหรับวาดเส้นทาง
Master	Vehicle_List	แปลงชื่อรถเป็น IMEI 15 หลัก เป็นคีย์หลักของทั้งระบบ
Master	Community_Zones	ชื่อสถานที่, ประเภท (ต้องเป็น polygon), จำกัดความเร็ว, พิกัดมุม, Looker_Geodata (WKT)
Master	Driver_Master / Driver_Assignment	รหัสและชื่อคนขับ ผูกกับรถรายเดือน (YYYY-MM) เพื่อตัดเกรดรายบุคคล
Analytical	Driver_Monthly_Score	คะแนนรายเดือนต่อรถ: จำนวนครั้ง, MaxSpeed, Score, Grade
Analytical	Driver_Safety_KPI	เกรด KPI รายบุคคล: Platinum ถึง Critical พร้อมสถานะ
Analytical	Driver_Risk_Flag	ผลวิเคราะห์เสี่ยง 3 เดือน: Avg3Month, RiskLevel, Remark
Analytical	Driver_Coaching_Plan	แผนโค้ชชิ่ง: พฤติกรรมหลัก, คำแนะนำ, Action Plan, วันติดตาม

จุดออกแบบที่สำคัญของชั้นนี้คือ Safe Write Protocol

ทุกการอัปเดตข้อมูลต้องสะสมในอาเรย์สองมิติแล้ว เขียนครั้งเดียวด้วย setValues ผ่าน Utils.writeSheet หรือ Utils.appendData เท่านั้น ห้ามเรียก setValue ในลูปเด็ดขาด เพราะทั้งช้าจนชนรันไทม์และเสี่ยงโศกนาฏกรรม นอกจากนี้ writeSheet ยังล้างเฉพาะแถวข้อมูลโดยไม่แตะแถวหัวตาราง เพื่อรักษา Schema ให้ AppSheet ที่จะมา Sync ในอนาคตไม่พัง กฎข้อนี้ถูกประกาศเป็นกฎหมายละเมิดในเอกสาร Forensic และถือเป็นหนึ่งในสิ่งที่นักพัฒนาคนต่อไปต้องเคารพ

3.4 ชั้นนำเสนอ: สองช่องทางแจ้งเตือนกับหนึ่งแผนที่

กลยุทธ์การแจ้งเตือนเลือกทางกลางที่ชาญฉลาด ระหว่างทางเลือกข้อแพ็คเกจ LINE เพิ่ม กับทั้ง LINE ไปใช้ Telegram ทั้งหมด ระบบเลือกส่งคู่ขนานโดยให้ Telegram

เป็นช่องทางเต็มรูปแบบที่แนบ HTML ลิงก์แผนที่รวมได้ครบ ส่วน LINE
 ส่งเฉพาะข้อความตัวอักษรโดยถอดแท็กลิงก์ออกและตัดข้อความที่ 4,800
 ตัวอักษรพร้อมประโยคนำให้ไปดู ฉบับเต็มทาง Telegram หรือ Google Sheets
 ส่วนการนำเสนอเชิงกราฟิกใช้ WebApp ของ GAS เองที่เรนเดอร์ Leaflet.js
 วาดเส้นทางหลายหมุด พร้อม Polygon เขตชุมชนสีแดงและ Popup
 แสดงความเร็ว-เวลาของทุกจุดที่คลิกได้ โดยรองรับทั้งพารามิเตอร์รูปแบบใหม่ (pts)
 และรูปแบบเก่า (lat,lng) เพื่อให้ลิงก์ใน Log เก่ายังเปิดได้

3.5 ทะเบียนโมดูลซอฟต์แวร์

โครงสร้างโค้ดถูกแบ่งสามกลุ่มตามหน้าที่ กลุ่ม core เป็นค่าคอนฟิก เครื่องมือร่วม
 และกระบวนการหลัก กลุ่ม services เป็นเอนจินเฉพาะทางตั้งแต่การดึงข้อมูลจนถึงระบบคะแนน
 และกลุ่ม controllers กับ ui เป็นจุด เชื่อมต่อกับผู้ใช้
 ตารางต่อไปนี้สรุปโมดูลหลักพร้อมเวอร์ชันที่ยืนยันแล้ว ซึ่งเป็นแผนที่ที่นักพัฒนาคนต่อไปควรใช้
 นำทางเมื่อต้องแก้ไข

ตารางที่ 3: ทะเบียนโมดูลหลักของระบบ (ย่อ)

โมดูล	เวอร์ชัน	บทบาท
Config.gs	V2	ค่าคอนฟิกกลาง: API URL, เกณฑ์ความเร็ว, ชื่อซีต, ระดับ KPI (Token ดึงจาก PropertiesService)
Utils.gs	V2	readSheet / writeSheet / appendData, isPointInPolygon, standardizeDate, calculateDistance (Haversine)
Validator.gs	V2	ตรวจสอบรูปแบบวันที่ เดือน และ IMEI ก่อนรัน Pipeline
MainProcess.gs	V2.8	Orchestrator รายวัน: ล็อก, ดึงข้อมูล, วิเคราะห์, สร้างลิงก์, บีบพิกัด, ส่งแจ้งเตือน
GpsService.gs	V2	fetchAll ขนาน + Retry แบบ JSON Fallback + Parser ทนข้อผิดพลาดแปรผัน
SpeedAnalyzer.gs	V2.9	State Machine ถนนปกติ (>90 km/h, ต่อเนื่องไม่น้อยกว่า 120 วินาที) เก็บเส้นทางพิกัด
CommunitySpeedService. gs	V2.9	State Machine เขตชุมชนแบบไม่หน่วงเวลา + Ray Casting + ย่อพิกัด 20 จุด
NotificationService.gs	V2.8	Telegram เต็มรูปแบบ, LINE ตัดลิงก์และตัดข้อความที่ 4,800 ตัวอักษร
DriverScoreService.gs	V2	รวม Log รายเดือนตัดคะแนนตามกฎหมายหลายชั้น ตัดเกรด A-D

โมดูล	เวอร์ชัน	บทบาท
DriverRiskService.gs / CoachingService.gs / DriverKPIService.gs	V2	วิเคราะห์เสี่ยง 3 เดือน, กฎข้อซึ่งตามลำดับความสำคัญ, KPI ต่อบุคคล
WebApp.gs	V2.9	doGet เรนเดอร์ Leaflet.js: เส้นทางหลายหมุด, Polygon, Popup, Backward Compatible
MainController.gs / PipelineController.gs / Menu.gs	V2	เมนูบนซีต: รันรายวัน, รันย้อนหลัง, Full Monthly Pipeline, Health Check

บทที่ 4 กฎธุรกิจและอัลกอริทึมสำคัญ

4.1 เกณฑ์จับความเร็วสองโหมด

ปรัชญาการออกแบบของระบบคือแยกโหนดการบังคับใช้ให้ชัดเจนบนถนนปกติให้ยืดหยุ่นเพื่อรองรับเหตุการณ์จริง ออกจากการเหยียบเร่งเล็กน้อยชั่วขณะ ด้วยเงื่อนไขความเร็วเกิน 90 km/h ต่อเนื่องอย่างน้อย 2 นาที แต่ในเขตชุมชนให้เข้มงวดสูงสุดโดยไม่มีการหน่วงเวลาเลย เพราะผู้กำหนดนโยบายมองว่าทุกวินาทีที่ขับเร็วในพื้นที่มีคนเดินผ่าน คือความเสี่ยงจริง ตารางต่อไปนี้จะเทียบกฎทั้งสองโหมดเคียงกัน

ตารางที่ 4: เปรียบเทียบกฎการจับความเร็วสองโหมด

ประเด็น	ถนนปกติ (SpeedAnalyzer)	เขตชุมชน (CommunitySpeedService)
ค่าเกณฑ์	เกิน 90 km/h (CONFIG.SPEED_THRESHOLD)	อ้างอิงซีต Community_Zones (ค่าเริ่มต้น 30 km/h)
การหน่วงเวลา	ต่อเนื่องไม่น้อยกว่า 120 วินาที ถึงบันทึก	ไม่มีการหน่วง ผิดปั๊บบันทึกทันที
การนับครั้ง	หนึ่ง Session ต่อช่วงที่เกินเกณฑ์ต่อเนื่อง	State Machine: ขับเร็วแช่นับ 1 ครั้ง ผ่อนแล้วเร่งใหม่นับ 2 ครั้ง
การตรวจพื้นที่	ไม่ตรวจพื้นที่ (ทุกถนนทั่วไป)	Ray Casting กับ Polygon ของแต่ละโซน
พิกัดที่เก็บ	เส้นทางเต็มย้อยเหลือ 15 จุดต่อรถต่อวัน	เส้นทางของเหตุการณ์ย้อยเหลือ 20 จุดต่อเหตุการณ์
ช่วงเวลาทำงาน	03:00 - 22:00 (GMT+7)	เดียวกัน แต่จับแยกตามโซนที่กำหนดในซีต

4.2 Ray Casting และข้อจำกัดที่ควรรู้

อัลกอริทึมตรวจสอบที่ใช้วิธีมาตรฐานคือลากเส้นสมมติแนวนอนจากจุดพิกัดของรถไปทางขวา แล้วนับว่าตัดขอบ Polygon กี่ครั้ง ถ้าจำนวนครั้งเป็นเลขคี่แปลว่าอยู่ข้างใน วิธีนี้แม่นยำและเร็วมากสำหรับ Polygon ธรรมดา อย่างไรก็ตามมีข้อจำกัดสองข้อที่ทีมควรทราบ ข้อแรกคืออัลกอริทึมไม่ตรวจสอบว่า Polygon ที่บ่อนเข้ามาถูกต้องหรือไม่ ถ้าผู้ใช้วาดเส้นตัดกันเอง (Self-intersecting) หรือใส่มุมน้อยกว่าสามจุด ผลลัพธ์อาจผิดโดยไม่มีการเตือน ข้อสองคือการเช็คคำตอบทุกจุด GPS กับทุกโซน หากจำนวนโซนเพิ่มเป็นสิบหรือร้อยในอนาคต ต้นทุนการคำนวณจะโต แบบเชิงเส้นต่อจำนวนจุด ควรเพิ่ม Bounding Box ด้านแรกเพื่อคัดโซนที่ไม่มีโอกาสเกี่ยวข้องออกก่อน

4.3 ระบบคะแนน หักแต้ม และเกรด

ทุกรถเริ่มเดือนด้วยคะแนนเต็ม 100 การหักคะแนนถูกออกแบบเป็นขั้นซ้อนกันตามความรุนแรง ผิดปกติหนึ่งครั้ง หัก 2 คะแนน ถ้าความเร็วสูงสุดเกิน 100 หักเพิ่มอีก 3 ถ้าเกิน 110 หักเพิ่ม 5 แทนชั้นก่อน และถ้าขับเร็วแช่เกิน 5 นาทีหักเพิ่มอีก 2 ผังเขตชุมชนหักหนักที่สุดคือครึ่งละ 5 คะแนนทันทีโดยไม่สนความเร็วส่วนเกิน คะแนนต่ำสุด ถูกยึดที่ 0 และตัดเกรดสี่ระดับ A (ตั้งแต่ 85), B (ตั้งแต่ 75), C (ตั้งแต่ 60) และ D (ต่ำกว่า 60)

ตารางที่ 5: กฎการหักคะแนนและการตัดเกรด

เงื่อนไข	ผลกระทบ	หมายเหตุ
ขับเร็วผิด 1 ครั้งบนถนนปกติ	-2 คะแนน	ค่าพื้นฐานต่อเหตุการณ์
ความเร็วสูงสุดเกิน 100 km/h	-3 คะแนนเพิ่ม	ถ้าเกิน 110 จะหัก 5 แทน
ความเร็วสูงสุดเกิน 110 km/h	-5 คะแนนเพิ่ม	แทนที่ชั้นเกิน 100
ขับเร็วแช่เกิน 5 นาที	-2 คะแนนเพิ่ม	จากระยะเวลาใน Log
ผิดในเขตชุมชน 1 ครั้ง	-5 คะแนน	ไม่สนระดับความเร็วส่วนเกิน
เกรด A / B / C / D	85 / 75 / 60 / ต่ำกว่า	คะแนนขั้นต่ำของเกรดนั้น

4.4 ความเสี่ยง KPI และแผนไค้ชิง

ระบบวิเคราะห์ความเสี่ยงมองย้อนหลังสามเดือนโดยจัดกลุ่มเป็น HIGH เมื่อได้เกรด D ในเดือนล่าสุด หรือได้เกรด C หรือ D ติดต่อกันสองเดือน และ MEDIUM เมื่อคะแนนเฉลี่ยสามเดือนต่ำกว่า 70 กลุ่มปกติไม่ถูกนำมา รายงานเพื่อให้ผู้จัดการโฟกัสเฉพาะรายที่ต้องแทรกแซง

จากนั้นเอนจินโค้ดซึ่งเลือกแผนตามกฎลำดับความสำคัญ จากบนลงล่าง กฎไหนจริงก่อนได้ทันที ได้แก่ ความเสี่ยงสะสมสูงให้โค้ชตัวต่อตัวพร้อมประเมินซ้ำใน 14 วัน ความเร็วรุนแรงเกิน 110 ให้อบรมความปลอดภัย 1 วันและประเมินซ้ำใน 30 วัน ผิดซ้ำเกิน 5 ครั้งให้บันทึกตักเตือน และติดตามรายสัปดาห์ และกรณีทั่วไปให้ทดสอบซ้ำจริงกับหัวหน้างาน สุดท้าย KPI รายบุคคลถูกแมปเป็นห้าระดับ ตั้งแต่ Platinum (95 ขึ้นไป) ถึง Critical (ต่ำกว่า 75) โดยระดับ Bronze ขึ้นไปถือเป็นสถานะเฝ้าระวังขึ้นไป

บทที่ 5 ผลการตรวจสอบ: จุดแข็งของระบบ

ก่อนเข้าสู่ข้อวิจารณ์ สมควรยอมรับจุดแข็งที่ทำให้ระบบนี้ยืนได้จริงในสนาม

จุดแข็งเหล่านี้ไม่ใช่เรื่องบังเอิญ

แต่มาจากการแก้ปัญหาซ้ำแล้วซ้ำเล่าจนกลายเป็นแบบแผนที่นักพัฒนาคนต่อไปควรสืบทอด

- **ประสิทธิภาพที่ออกแบบรอบข้อจำกัดแพลตฟอร์ม:** การย้ายจาก Fetch ที่ละคันไปเป็น fetchAll ขนานลดเวลาจาก 6 นาทีเหลือไม่ถึง 20 วินาที พร้อม Retry อัตโนมัติเมื่อ API ตอบว่าง ซึ่งแก้ทั้งปัญหาเวลาและความไม่เสถียร ของผู้ให้บริการในคราวเดียว
- **ความสมบูรณ์ของข้อมูล:** Safe Write Protocol ที่บังคับเขียนแบบ setValues ครั้งเดียวและล้างเฉพาะแถวข้อมูล ทำให้ทั้งป้องกันการชนโควตาและรักษาหัวตารางสำหรับ AppSheet ไปพร้อมกัน
- **การทนทานต่อข้อมูลสกปรก:** Utils.standardizeDate กวาดวันที่ทุกรูปแบบ (UK/ISO/ตัวแปร Sheets) ให้เป็นมาตรฐานเดียว แก้ปัญหาคลาสสิกของ Google Sheets ที่ฟอร์แมตต่างกันตามภูมิภาคผู้ใช้
- **การแยกความรับผิดชอบชัดเจน:** โมดูล 18 ไฟล์ถูกจัดเป็น core / services / controllers ที่หน้าที่ไม่ซ้อนทับกัน ทำให้แก้บั๊กแจ้งเตือนโดยไม่กระทบฝั่งวิเคราะห์ที่ได้จริง (พิสูจน์แล้วตอนอัปเดต V2.8 กับ V2.9 แยกสายกัน)
- **กลยุทธ์ช่องทางแจ้งเตือนที่เข้าใจต้นทุน:** การแยก Telegram เต็มรูปแบบกับ LINE แบบประหยัดโควตา ทำให้ไม่ต้องจ่ายค่าแพ็คเกจ แต่ยังได้ข้อมูลครบที่ผู้ดูแลและได้แจ้งเตือนรวดเร็วที่ผู้ใช้งาน
- **วินัยการเก็บหลักฐาน:** Forensic Export ฉบับนี้เองคือหลักฐานว่าทีมมีวัฒนธรรมบันทึกการตัดสินใจ ข้อผิดพลาด และกฎห้ามละเมิด ซึ่งเป็นสิ่งที่องค์กรส่วนใหญ่ขาดและเป็นสาเหตุที่งานต่อ ยอดมักทำซ้ำความผิดเดิม

นัยเชิงบริหารของจุดแข็งเหล่านี้คือต้นทุนการดำเนินงานต่อเนื่องต่ำมาก

ระบบไม่มีค่าเช่าเซิร์ฟเวอร์ ไม่มีฐานข้อมูลที่ต้องจ่าย

และการบำรุงรักษาหลักอยู่ที่การติดตามโควตากับการปรับกฎธุรกิจซึ่งทำได้ผ่านซีตหรือค่าคอนฟิกโดยตรง หากองค์กรมีแผนขยายจำนวนรถหรือจำนวนโซน ต้นทุนส่วนเพิ่มในระยะสั้นยังเป็นศูนย์ จนกว่าจะชนขีดจำกัดที่วิเคราะห์ไว้ในบทถัดไป

บทที่ 6 จุดอ่อนและความเสี่ยงเชิงสถาปัตยกรรม

บทนี้คือหัวใจของการตรวจสอบในฐานะที่ปรึกษาเทคนิค ทุกข้อที่กล่าวถึงอ้างอิงเหตุการณ์ที่อ่านได้จากโค้ดจริง ในเอกสาร Forensic โดยจัดกลุ่มเป็นหกประเด็นตามความรุนแรง และปิดท้ายด้วยเมทริกซ์ความเสี่ยงรวม ข้อสังเกตส่วนใหญ่ไม่ใช่บั๊กที่กำลังทำให้ระบบพังในวันนี้ แต่เป็นเหมือนรอยร้าวที่จะขยายตัวเมื่อปริมาณข้อมูล จำนวนรถ หรือจำนวนผู้ใช้เพิ่มขึ้น การแก้ตอนนี้ถูกกว่าการแก้ตอนที่ข้อมูลสะสมมาหลายปีมาก

6.1 Google Sheets เป็นฐานข้อมูลเดียวของระบบ

การเลือก Sheets เป็นฐานข้อมูลให้ต้นทุนศูนย์และสะดวกมากในช่วงเริ่มต้น แต่มีขีดจำกัดโครงสร้างสามชั้น ชั้นแรกคือขีดจำกัดจำนวนเซลล์ 10 ล้านเซลล์ต่อสเปรดชีต ซีต Speeding_Log ที่เติมข้อมูลรายวันแบบ Append โดยไม่มีแผนเก็บถาวรจะเข้าใกล้ขีดจำกัดนี้เงียบ ๆ และเมื่อชนแล้วระบบจะหยุดเขียนทันทีทั้งระบบ ชั้นที่สองคือ ต้นทุนการอ่าน DriverScoreService อ่าน Speeding_Log ทั้งซีตทุกครั้งทีรัน Pipeline รายเดือน ซึ่งเป็นความซับซ้อน แบบ $O(n)$ ที่โตตามประวัติสะสม ยิ่งระบบรันนาน การคำนวณยิ่งช้าและยิ่งเสี่ยงชนรันไหม ชั้นที่สามคือการค้นหา ที่ไม่มีดัชนี ทุกการ Join ทำด้วยการวนลูปเทียบข้อความซึ่งแพงเมื่อข้อมูลโต

แนวทางบรรเทาที่ไม่ต้องยกเครื่องทั้งหมด คือแบ่งซีต Log ตามปี (เช่น Speeding_Log_2026) และให้ตัวคำนวณ รายเดือนอ่านเฉพาะซีตของปีที่น่าสนใจ พร้อมกำหนดนโยบายย้ายข้อมูลเก่ากว่า 24 เดือนไปไว้สเปรดชีต Archive แยกต่างหาก หากระบบโตถึงจุดที่ Sheets ไม่ไหวจริง ๆ ทางเลือกระยะยาวคือย้ายเฉพาะตาราง Transactional ไปยังฐานข้อมูลที่เหมาะสม โดยให้เลเยอร์ Services ยังคงเป็นแพลตฟอร์มเดิมเพื่อรักษาการลงทุนเดิมให้มากที่สุด

6.2 การเขียนข้อมูลแบบล้างก่อนเขียน (Destructive Write)

Utils.writeSheet ทำงานสามขั้นตอนคือล้างข้อมูลเก่าทั้งหมดออกก่อน แล้วจึงเขียนชุดใหม่ลงไป ขั้นตอนทั้งสองไม่ใช่ปฏิบัติการเดียวกัน (ไม่มี Atomicity) ถ้าสคริปต์ขัดข้องกลางทางด้วยเหตุผลใดก็ตาม เช่น โควตาหมด เน็ตเวิร์ก ล่วงชั่วคราว หรือรันไหม้หมด ตารางจะค้างอยู่ในสถานะว่างเปล่าและข้อมูลอาจสูญหายถาวร

เพราะปัจจุบันไม่มีกลไกสำรองที่กู้คืนได้นอกจาก Version History ของ Sheets ซึ่งต้องกู้เองด้วยมือ และหาก มี AppSheet หรือผู้ใช้เขียนทับในช่วงนั้น ความเสียหายจะขยายออกไปอีก

รูปแบบที่แนะนำคือ Copy-then-Swap ก่อนเขียนทับตารางใด ให้คัดลอกค่าปัจจุบันทั้งหมดไปเก็บในแท็บ สำรอง (เช่น ชื่อเดียวกันต่อท้ายด้วย _bak) หรือเขียนไปยังแท็บ Staging แล้วค่อยสลับเนื้อหา ต้นทุนเพิ่มเพียง ไม่กี่วินาทีต่อรอบ แต่เปลี่ยนการกู้คืนจากเรื่องยากให้เป็นเรื่องง่ายได้ อีกทางเลือกเสริมคือตั้ง Trigger รายวัน คัดลอกสเปรดชีตทั้งไฟล์ไปยังโฟลเดอร์สำรองผ่าน DriveApp เก็บย้อนหลัง 7 วัน ซึ่งครอบคลุมความเสี่ยง ทุกตารางในครั้งเดียว

6.3 ความล้มเหลวแบบเงียบ (Silent Failure)

ระบบมีจุดที่หายเงียบอย่างน้อยสองจุดที่สำคัญ จุดแรกคือใน MainProcess เมื่อ waitLock รอเกิน 30 วินาที สคริปต์จะคืนค่า 0 ออกไปเฉย ๆ โดยไม่บันทึกหรือแจ้งใคร ผู้กดรันจะเห็นข้อความว่าพบเหตุการณ์ 0 รายการ ซึ่งดูเหมือนวันนี้ไม่มีเหตุการณ์ ทั้งที่ความจริงคืองานยังไม่ถูกรันเลย ในบริบทความปลอดภัยฟลิต การเข้าใจผิด ลักษณะนี้อันตรายเพราะทุกคนคิดว่าการควบคุมกำลังทำงาน จุดที่สองคือบล็อก catch ใน GpsService และ NotificationService ที่เขียน console.error ไว้เท่านั้น ซึ่งไม่มีใครมองเห็นเว้นแต่จะเปิด Logs ด้วยตนเอง ความผิดพลาดของการแจ้งเตือนจึงกลายเป็นความเงียบที่ไม่มีใครรู้ว่าพนักงานไม่ได้รับสรูปของวันนี้

ทางแก้มีต้นทุนต่ำมาก เพิ่มฟังก์ชันรายงานข้อผิดพลาดกลางที่ส่งข้อความเข้า Telegram ผู้ดูแล (แยกกลุ่ม จากกลุ่มแจ้งเตือนปกติได้) พร้อมชื่อฟังก์ชันและข้อความ Error แล้วเรียกในทุก catch ที่สำคัญ ทั้งยังควรเปลี่ยน พฤติกรรม waitLock จากการคืน 0 เป็นการแจ้งเตือนว่ามีงานอื่นกำลังรันและขอให้ลองใหม่ภายหลัง เพื่อตัด ความกำกวมระหว่าง ไม่มีเหตุการณ์ กับ ยังไม่ได้ประมวลผล

6.4 ความปลอดภัยของ WebApp และการจัดการความลับ

WebApp ปัจจุบันเปิดอ่านข้อมูลได้จากลิงก์โดยตรงโดยไม่มีการยืนยันตัวตน และพารามิเตอร์อย่างชื่อรถ (v) กับชื่อสถานที่ (z) ถูกแทรกเข้า HTML ด้วย Template String โดยไม่ผ่านการ Escape แปลว่าใครก็ตามที่สร้างลิงก์ ปลอมที่ใส่สคริปต์ในพารามิเตอร์ได้ อาจยิง XSS ใส่ผู้ที่เปิดลิงก์นั้นได้ ความเสี่ยงนี้ยังไม่เร่งด่วนเพราะลิงก์จริง ทั้งหมดถูกสร้างจากระบบ แต่การปิดช่องนี้ง่ายมากเมื่อเทียบกับผลที่ได้ ข้อเสนอคือ Escape ทุกค่าที่แทรกลง HTML ด้วยฟังก์ชันกันชนก่อนเรนเดอร์ และพิจารณาเพิ่มโทเคนตรวจสอบง่าย ๆ ในพารามิเตอร์เพื่อให้ลิงก์ที่ไม่ได้ออกจากระบบ เปิดไม่ได้ นอกจากนี้ค่า TELEGRAM_CHAT_ID

และ LINE_TARGET_ID ยัง Hardcode อยู่ในโค้ด ควรย้ายลง PropertiesService ให้ครบตามแนวทางเดิมที่ทำไว้กับ Token เพื่อให้การแชร์ซอร์สโค้ดภายในทีมไม่นำไปสู่การปลอมแปลง การส่งข้อความเข้ากลุ่มแจ้งเตือน

6.5 ข้อจำกัดโควตาของแพลตฟอร์มที่ยังไม่มีตัวตรวจ

ระบบทำงานอยู่บนโควตาหลายชั้นที่ทุกชั้นเคยสร้างปัญหามาแล้วอย่างน้อยหนึ่งครั้ง ชั้น LINE มีข้อจำกัด ที่แรงที่สุดเพราะบัญชีฟรีของ Messaging API ให้ Push ได้ราว 200 ข้อความต่อเดือน ปัจจุบันการตัดลิงก์ช่วยเรื่อง ความยาวข้อความ แต่ยังไม่มีส่วนนับจำนวนข้อความที่ใช้ไป เมื่อโควตาหมดระบบจะพยายามส่งแล้วล้มเหลวเงียบ ๆ ตามพฤติกรรมในข้อ 6.3 ชั้น URL Fetch ของ GAS มีโควตารายวันซึ่งปัจจุบันห่างไกล แต่ควรเฝ้าถ้าฟลีตโตเป็นหลักร้อยละ และชั้นรันไทม์ 6 นาทีที่เคยเป็นปัญหา ปัจจุบันใช้ไม่ถึง 20 วินาทีจึงยังสบาย ข้อเสนอคือเพิ่มการอ่านค่าตอบกลับจาก LINE API ที่มีสถานะโควตา (x-line-remaining-quota) มาบันทึกไว้ในชีตสถานะ และแจ้งเตือนเมื่อเหลือน้อยกว่า ค่ากำหนด ซึ่งเขียนไม่กี่บรรทัดแต่ปิดประตูความเสี่ยงที่เคยเกิดจริง

6.6 คุณภาพโค้ดและความเปราะบางของ Schema

ประเด็นสุดท้ายเป็นกลุ่มข้อสังเกตระดับโค้ดที่รวมกันแล้วสร้างต้นทุนการบำรุงรักษาระยะยาว ประเด็นแรก คือการอ้างตำแหน่งคอลัมน์ด้วยเลขดัชนี เช่น s[8] หรือ row[8] ใน MainProcess และการพึ่งคอลัมน์ลำดับที่ 9 สำหรับลิงก์แผนที่รวม ถ้าใครแทรกหรือสลับคอลัมน์ในชีต ระบบจะเขียนข้อมูลผิดตำแหน่งโดยไม่ Error ควรย้ายไปอ้างชื่อคอลัมน์จากค่าคอนฟิกแบบเดียวกับที่อ่านข้อมูล ประเด็นที่สองคือลอจิกระยะเวลาของ SpeedAnalyzer วิวัด ระยะเวลาจากจุดแรกถึงจุดสุดท้ายของช่วงเกินเกณฑ์ หากอุปกรณ์หลุดสัญญาณเป็นช่วง ๆ ระยะเวลาอาจถูกประเมิน สูงเกินจริงหรือต่ำเกินจริง และพฤติกรรมเหยียบเกิน-ผ่อนพอดีชอบ 90 จะทำให้ Session ถูกทิ้งทั้งชุดถ้าไม่ครบ 2 นาที ซึ่งเป็นเหตุที่ควรบันทึกสถิติว่ามี Session ถูกทิ้งเท่าไรเพื่อตรวจสอบความแม่นยำของการนับ ประเด็นที่สาม คือยังไม่มีชุดทดสอบอัตโนมัติแม้แต่ชุดจำลองพื้นฐาน ทั้งที่มีสคริปต์ polygon_logic_simulation เป็นแบบแผนอยู่แล้ว ควรขยายเป็นชุดทดสอบกฎคะแนนและ State Machine เพราะกฎกลุ่มนี้คือหัวใจเชิงนโยบายที่ผลิตแล้วกระทบคนจริง

ตารางที่ 6: เมทริกซ์ความเสี่ยงเชิงสถาปัตยกรรม

ความเสี่ยง	ผลกระทบหากเกิดจริง	โอกาส	ระดับ	แนวทางจัดการหลัก
Sheets ขนชีตจำกัดเซลล์ / ซ้ำ	ระบบหยุดเขียนทั้งระบบ	กลาง	สูง	แบ่งชีตรายปี + Archive ข้อมูลเก่า

ความเสี่ยง	ผลกระทบหากเกิดจริง	โอกาส	ระดับ	แนวทางจัดการหลัก
เขียนทับลั้กลางทาง	สูญเสีย Log ทั้งตาราง	ต่ำ	สูง	Copy-then-Swap + สำรองรายวัน
งานล้มเงียบ (Silent Failure)	ผู้จัดการคิดว่าระบบทำงานทั้งที่ไม่ได้รับ	กลาง	สูง	ช่องทางแจ้ง Error เข้า Telegram ผู้ดูแล
โควตา LINE หมดกลางเดือน	ผู้ใช้ LINE ไม่ได้รับสรุปรายวัน	สูง	กลาง	ตรวจสอบสถานะโควตา + เกณฑ์แจ้งเตือน
XSS / การเดาลิงก์ WebApp	ข้อมูลหลุดและเส้นทางรั่ว	ต่ำ	กลาง	Escape พารามิเตอร์ + โทเคนในลิงก์
Schema ซิดถูกแก้ (ดัชนี 8/9)	ข้อมูลเขียนผิดคอลัมน์เงียบ ๆ	กลาง	กลาง	อ้างอิงกลับด้วยชื่อจากค่าคอนฟิก
ระยะเวลา Session ผิดเพี้ยน	การนับครั้ง/คะแนนเบี่ยงจากจริง	กลาง	กลาง	ตรวจสอบช่วงเวลาระหว่างจุด + สถิติทั้ง
Playback URL เดามิด (กล่อง MDVR)	พีเจอร์ตรวจสอบเหตุการณ์ใช้ไม่ได้	สูง	กลาง	แผนทดสอบ 3 ชั้นในบทที่ 7

บทที่ 7 Playback และการตรวจสอบเหตุการณ์ (Incident Investigation)

7.1 ทำไม Playback คือหัวใจของ Fleet Safety จริง

ความต้องการที่ชัดที่สุดของฝั่งธุรกิจในขณะนี้คือการเห็นภาพเหตุการณ์ ณ วินาทีที่ระบบแจ้งเตือนว่าขับเร็ว ไม่ใช่ภาพสดในปัจจุบัน เหตุผลเชิงหน้าที่สมเหตุสมผลอย่างยิ่ง เพราะกระบวนการตรวจสอบเหตุการณ์ (Incident Investigation) ต้องตอบคำถามสามข้อพร้อมกัน ข้อแรกคือระบบแจ้งถูกหรือไม่ (ความเร็วจริง บริบทถนน สภาพแวดล้อม) ข้อสองคือพฤติกรรมของคนขับเป็นอย่างไร (ท่าทาง การหันหน้า การถือโทรศัพท์ จากกล้องในห้องโดยสาร) ข้อสามคือบริบทเสริมอื่น (ผู้เดินเท้า สัตว์ ยานพาหนะคู่กรณี สภาพแสง) ภาพวิดีโอย้อนหลังจากกล่อง MDVR คือหลักฐานชั้นเดียวที่ตอบทั้งสามข้อพร้อมกันได้ และเป็นสะพานเชื่อมข้อมูล GPS ที่บอกว่า เกิดอะไรขึ้นที่ไหน เมื่อไร กับหลักฐานภาพที่บอกว่า เกิดขึ้นอย่างไร

ในทางเทคนิค ระบบปัจจุบันมีข้อมูลครบทุกองค์ประกอบที่ต้องใช้แล้ว ทั้งเวลาเริ่มและเวลาสิ้นสุดของเหตุการณ์ ใน Speeding_Log และ Community_Alerts ชื่อรถที่แมปกับทะเบียนกล่องได้ และลิงก์แจ้งเตือนที่ส่งถึงผู้จัดการทุกวัน สิ่งที่ยังขาดจึงไม่ใช่ข้อมูล แต่เป็นชิ้นส่วนกลางที่หยิบเวลาเหตุการณ์ไปเปิดวิดีโอช่วงนั้นให้โดยอัตโนมัติ

รายละเอียดการออกแบบชิ้นส่วนนี้คือหัวใจของบทนี้

7.2 สถาปัตยกรรมการเชื่อมต่อกล้องปัจจุบัน (CMSV6 / MDVR)

จากเอกสารขั้นตอนการเชื่อมต่อ API กล้องที่ทีมใช้อยู่ กระบวนการเป็นสองขั้น
ขั้นแรกยิงคำขอเข้าสู่ระบบ ไปที่ StandardApiAction_login.action ด้วยพารามิเตอร์ account
และ password เพื่อรับค่า jsession ขั้นที่สองนำค่า jsession ไปต่อท้ายหน้าเล่นวิดีโอ
RealPlayVideo.html พร้อม PlateNum ของรถ เช่น 2มด-6206
เพื่อเปิดภาพสดจากกล้องทั้งหน้ารถและในห้องโดยสาร
โดยภาพสดระบุพิกัดและความเร็วปัจจุบันบนหน้าจอด้วย เอกสาร Forensic ระบุตรงกันว่า
CameraService.getPlaybackUrl ที่เตรียมไว้ในโค้ดเป็นการคาดเดาโครงสร้าง URL
จากสถาปัตยกรรม CMSV6 ซึ่งต้องได้รับการทดสอบหรือขอเอกสาร API จากผู้ขายก่อนใช้จริง

ตารางที่ 7: คุณสมบัติของลิงก์กล้องในรูปแบบปัจจุบัน เทียบกับความต้องการ Incident Investigation

ประเด็น	สิ่งที่มีอยู่ตอนนี้ (Realtime Link)	สิ่งที่ Incident Investigation ต้องการ
ช่วงเวลาให้เห็น	ภาพสดขณะเปิดลิงก์เท่านั้น	วิดีโอช่วงเวลาเกิดเหตุ เช่น เวลาเริ่มลบ 1 นาที ถึงสิ้นสุดบวก 1 นาที
การเชื่อมโยงเหตุการณ์	ผู้ใช้ต้องหาเวลาเองจาก Log	ลิงก์ต่อเหตุการณ์ที่ฝั่งเวลาเริ่ม-สิ้นสุดไว้แล้ว
อายุการใช้งานลิงก์	jsession มีอายุสั้นและหมดอายุทั้ง	ลิงก์ที่สร้างสดตอนคลิก (On-demand) จึงไม่ตาย
การยืนยันตัวตน	รหัสผ่านฝังใน URL และอยู่ในประวัติเบราว์เซอร์	ระบบกลางถือ Credential แทนผู้ใช้ ไม่ปล่อยรหัสผ่านลงลิงก์
การระบุรถ	PlateNum แบบทะเบียนจริง (เช่น 2มด-6206)	แมปชื่อรถใน Vehicle_List กับ PlateNum ของกล้องโดยอัตโนมัติ

7.3 ช่องว่างเชิงออกแบบและความเสี่ยงที่ต้องปิด

เมื่อวางสิ่งที่มีกับสิ่งที่ต้องการคู่กันตามตารางที่ 7 ช่องว่างจะชัดเจนสามช่อง ช่องแรกคือเวลา
ลิงก์ปัจจุบันเป็น Realtime ซึ่งใช้เฝ้าระวังได้แต่ใช้สืบสวนไม่ได้
เพราะเหตุการณ์ผ่านไปแล้วตั้งแต่วินาทีที่ข้อความแจ้งเตือนถูกพิมพ์ ช่องที่สองคืออายุ Session
ค่า jsession ที่ได้จากการ Login มีอายุจำกัด การสร้างลิงก์ล่วงหน้าเก็บใน Log จึงไม่
ปลอดภัยเชิงเวลา ต้องสร้าง Session สดเมื่อผู้ใช้คลิกเท่านั้น ช่องที่สามคือความปลอดภัย
การวางรหัสผ่านใน URL ทำให้ Credential ตกค้างในประวัติเบราว์เซอร์และบันทึกของเซิร์ฟเวอร์
และถ้าลิงก์ถูกส่งต่อ ผู้รับจะเห็นรหัสผ่านทั้งบัญชี ของผู้ให้บริการกล้อง

ทั้งสามช่องทางนี้ชี้ไปทางเดียวกันว่าต้องมีบริการกลางตัวเล็กที่ทำหน้าที่นายหน้าเปิดวิดีโอแทนผู้ใช้

7.4 ข้อเสนอการออกแบบ: Playback Deep-Link ผ่านบริการนายหน้า

โครงสร้างที่แนะนำคือเพิ่มจุดปลาย (Endpoint) หนึ่งตัวบน WebApp เดิมของ GAS

ทำหน้าที่เป็นนายหน้า สี่ขั้นตอน ขั้นที่หนึ่งผู้ใช้คลิกลิงก์ เช่น

doGet?action=playback&v;=บธ-7049&ts;=2026-02-27 16:51:48

ที่แนบมากับข้อความแจ้งเตือนหรือขีด Log ขั้นที่สองบริการนายหน้า Login เข้า CMSV6

ฝั่งเซิร์ฟเวอร์ด้วย Credential ที่เก็บใน PropertiesService เพื่อรับ jsession ใหม่เสมอ

(กันปัญหาอายุ Session) ขั้นที่สามแปลงชื่อรถเป็น PlateNum ผ่านคอลัมน์ใหม่ใน Vehicle_List

และคำนวณช่วงเวลาเล่นย้อนหลังจากเวลาเหตุการณ์ เช่น ก่อนเริ่ม 1 นาทีถึงหลัง สิ้นสุด 2 นาที

ขั้นที่สี่เปลี่ยนเส้นทาง (Redirect) ไปยังหน้า Playback ของผู้ให้บริการที่ฝั่ง jsession, PlateNum และช่วงเวลาครบ ผู้ใช้ไม่เคยเห็นรหัสผ่านตลอดกระบวนการ

- **ขั้นที่ 1 - จุดคลิก:** เพิ่มลิงก์ ดูวิดีโอย้อนหลัง ต่อท้ายแต่ละเหตุการณ์ใน Telegram, Sheets และ Popup ของแผนที่ โดยฝั่งชื่อรถและเวลาเริ่มของเหตุการณ์ในพารามิเตอร์
- **ขั้นที่ 2 - Login สด:** บริการนายหน้ายิง StandardApiAction_login ฝั่งเซิร์ฟเวอร์ทุกครั้งที่มีคลิก ใช้ Credential จาก PropertiesService (เพิ่ม CAM_ACCOUNT / CAM_PASSWORD)
- **ขั้นที่ 3 - เตรียมพารามิเตอร์:** แมปรถกับ PlateNum ผ่านคอลัมน์ใหม่ใน Vehicle_List และตัดช่วงเวลาเล่น จากเวลาเหตุการณ์ (Buffer ปรับได้ในค่าคอนฟิก)
- **ขั้นที่ 4 - Redirect:** ส่งต่อไปหน้า Playback ของ CMSV6 พร้อม jsession, PlateNum, ช่วงเวลา และภาษา โดยชื่อโดเมนและเส้นทางจริงต้องยืนยันจากแผนทดสอบในหัวข้อถัดไป

ข้อควรย้าเชิงวิศวกรรม: เอกสาร Forensic ยืนยันว่าโครงสร้าง URL Playback ปัจจุบันเป็นการอนุมานจาก สถาปัตยกรรม CMSV6 ไม่ใช่ข้อเท็จจริงที่พิสูจน์แล้ว ดังนั้นการนำโครงสร้างข้างต้นไปเขียนควรหยุดที่ขั้นทดสอบ จนกว่าจะได้ URL จริงที่เล่นวิดีโอช่วงเวลาที่เราได้ หากทดสอบแล้วไม่สำเร็จ ทางเลือกถัดไปคือขอเอกสาร API Playback จากผู้ขายกล้องโดยตรง หรือใช้ระบบดูย้อนหลังบนหน้าเว็บของผู้ขายควบคู่กับการฝังเวลาในลิงก์ค้นหา เบื้องต้นก่อน

7.5 แผนทดสอบสามขั้นก่อนพัฒนาจริง

เพื่อลดความเสี่ยงก่อนเขียนโค้ดตามโครงสร้าง ควรรันแผนทดสอบสั้น ๆ

สามขั้นที่ใช้เวลารวมไม่ถึงวันเดียว ขั้นที่หนึ่งทดสอบอายุ Session ยิง Login แล้วจับเวลาว่า

jsession ใช้งานได้นานเท่าใด และ Login ซ้อนกัน บ่อยแค่ไหนได้โดยไม่โดนตัด (ข้อมูลนี้กำหนดกลยุทธ์ Cache Session ในอนาคต) ขั้นที่สองทดสอบ URL Playback ด้วยตนเอง โดยลองรูปแบบพารามิเตอร์เวลาหลายแบบกับรถทดสอบหนึ่งคันที่เคยเกิดเหตุการณ์จริง เพื่อพิสูจน์ว่า เล่นย้อนหลังตามช่วงเวลาได้จริง ขั้นที่สามทดสอบการแมปชื่อรถ เทียบชื่อรถใน Vehicle_List กับ PlateNum ที่ระบบกล้องรู้จักว่าตรงกันรึอย่าปอร์เซ็นต์หรือไม่ และจัดรูปแบบให้เป็นมาตรฐานเดียว เมื่อสามขั้นนี้ผ่าน การพัฒนาโครงสร้างเต็มจึงมีความมั่นใจสูง และตอบโจทย์ Incident Investigation ได้ครบวงจรตั้งแต่ข้อความแจ้งเตือน จนถึงภาพเหตุการณ์

บทที่ 8 แผนพัฒนาต่อและลำดับความสำคัญ

8.1 งานค้างเดิมที่วางไว้แล้ว

เอกสาร Forensic ระบุงานค้างสามชิ้นที่ควรต่อยอดโดยตรง ชิ้นแรก Looker Studio โค้ดสร้างพารามิเตอร์ ds0 (วันที่ ชื่อรถ สถานที่) ถูกเขียนเตรียมไว้ใน MainProcess แล้วแต่ถูก Comment ไว้ตามคำสั่งผู้ใช้ที่ต้องการปรับ Dashboard ใหม่ เมื่อ Dashboard ใหม่พร้อม การเปิดใช้เป็นเรื่องของการถอด Comment และทดสอบลิงก์ ขั้นที่สอง AppSheet โครงสร้าง Schema รองรับ Sync แล้ว สิ่งที่ต้องทดสอบคือ Column Index จริงเมื่อเปิดใช้งานจริง ซึ่งผูกกับข้อเสนอในข้อ 6.6 เรื่องการอ้างอิงคอลัมน์ด้วยชื่อ ทั้งสองงานควรทำพร้อมกันเพื่อให้ผลการทดสอบ คุ่มค่า ขั้นที่สามคือกล้อง MDVR ที่บทที่ 7 วิเคราะห์ลงลึกไว้แล้ว

8.2 งานเพิ่มเติมจากผลการตรวจสอบ

นอกจากงานค้างเดิม ผลตรวจในบทที่ 6 ชี้งานพื้นฐานสี่ชิ้นที่ควรแทรกเข้ามาก่อนฟีเจอร์ใหม่ใด ๆ ได้แก่ ระบบสำรองข้อมูลรายวันแบบอัตโนมัติ ช่องทางแจ้ง Error เข้า Telegram ผู้ดูแล ตัวตรวจสถานะไควตา LINE และการปรับการอ้างอิงคอลัมน์ให้อิงชื่อ งานทั้งสี่ไม่มีขึ้นใดเกินเวลาเกินสองวันทำงาน แต่รวมกันแล้วยกระดับความน่าเชื่อถือของระบบจากระดับ ทำงานได้ดีวันนี้ไปสู่ระดับ รู้ทันทีเมื่อเกิดปัญหาและกู้คืนได้เสมอ ซึ่งเป็น คุณสมบัติที่ระบบควบคุมความปลอดภัยควรมีมากกว่าฟีเจอร์ใหม่อื่น

8.3 ตารางลำดับความสำคัญรวม

ตารางที่ 8: แผนงานแนะนำจัดตามลำดับความสำคัญ

ลำดับ	งาน	เป้าหมายที่วัดได้	ประมาณการ
P0	ทดสอบ Playback ตามแผน 3 ชั้น (บทที่ 7.5)	รู้จริงว่า URL Playback ใช้ได้หรือต้องขอ API จากผู้ขาย	0.5-1 วัน
P0	สำรองข้อมูลรายวัน (คัดลอกสเปรดชีตเก็บ 7 วัน)	กู้คืนข้อมูลย้อน 7 วันได้ภายใน 10 นาที	0.5 วัน
P0	ช่องทางแจ้ง Error เข้า Telegram ผู้ดูแล + แก่ Silent waitLock	ทุกความล้มเหลวสำคัญถึงมือผู้ดูแลภายใน 1 นาที	1 วัน
P1	ตรวจโควตา LINE และเกณฑ์แจ้งเตือนเมื่อเหลือน้อย	ไม่มีวันที่สรุปหายเพราะโควตาหมดอีก	0.5 วัน
P1	เพิ่มคอลัมน์ PlateNum ใน Vehicle_List + Playback Deep-Link	คลิกจากแจ้งเตือนเห็นวิดีโอช่วงเหตุการณ์ได้	2-3 วัน
P1	เปิดใช้ Looker Studio (ds0) หลังปรับ Dashboard เสร็จ	ผู้บริหารเปิดรายงานเดือนจากลิงก์เดียว	1-2 วัน
P1	ทดสอบ AppSheet Sync คู่กับการอ้างอิงคอลัมน์ด้วยชื่อ	ฟอร์มมือถือใช้งานได้โดย Schema ไม่พัง	1-2 วัน
P2	แบ่งซีต Log รายปี + นโยบาย Archive 24 เดือน	ปริมาณเซลล์ต่อสเปรดชีตคงที่ไม่โตไม่จำกัด	2-3 วัน
P2	ชุดทดสอบจำลองกฎคะแนน/State Machine	เปลี่ยนกฎได้อย่างมั่นใจโดยไม่ต้องรอเดือนจริง	2-3 วัน
P2	ประเมินย้ายตาราง Transactional ออกจาก Sheets (ถ้าฟลิตขยาย)	แผนย้ายมีขั้นตอนชัดพร้อมต้นทุนที่เยียบ	1 สัปดาห์

ตรรกะของการจัดลำดับนี้คืองาน P0

ล้วนลดความเสี่ยงแบบถาวรด้วยค่าใช้จ่ายต่ำมากและไม่เปลี่ยนพฤติกรรม ระบบเดิม งาน P1 เป็นการส่งมอบคุณค่าใหม่ที่ผู้ใช้จับต้องได้โดยอิงข้อเท็จจริงจาก P0 และงาน P2 เป็นการเตรียมสถาปัตยกรรมรองรับการเติบโตซึ่งควรทำเมื่อสัญญาณความช้าหรือขนาดข้อมูลเริ่มชัดเจน ไม่ใช่ทำล่วงหน้าโดยไม่มี ความจำเป็น เมื่อทำตามลำดับนี้ ทีมจะได้ทั้งความมั่นคง פיเจอร์ตรวจสอบเหตุการณ์ครบวงจร และเส้นทางขยายตัว ที่ไม่ต้องยกเครื่องใหม่ทั้งหมด

บทที่ 9 สรุปและข้อเสนอแนะเชิงปฏิบัติ

Fleet Safety Intelligence V2 เป็นระบบที่สร้างเสร็จจริง ใช้งานจริง และพิสูจน์ความสำเร็จมาแล้ว ด้วยทีมเล็กบนแพลตฟอร์มที่ไม่มีค่ารายเดือน ซึ่งเป็นสิ่งที่ควรชื่นชม จุดแข็งหลักอยู่ที่ประสิทธิภาพของ Pipeline ความเข้าใจข้อจำกัดแพลตฟอร์มอย่างลึกซึ้ง และวินัยการเก็บเอกสารหลักฐาน จุดที่ต้องเร่งเสริมคือความน่าเชื่อถือ เชิงปฏิบัติการ (สำรองข้อมูล การแจ้งเตือนความผิดพลาด การเฝ้าไควตา) ความปลอดภัยของ WebApp และการต่อ สะพาน Playback ที่เชื่อมข้อมูล GPS กับหลักฐานภาพวิดีโอตามความต้องการเชิงนโยบายที่ชัดเจนที่สุดขององค์กร

- **1. เริ่มแผนทดสอบ Playback สามขั้นทันที**
เพราะเป็นงานครึ่งวันที่กำหนดชะตาฟีเจอร์สำคัญที่สุดขององค์กร และตัดสินใจจะเขียน Deep-Link เองหรือขอ API เอกสารจากผู้ขาย
- **2. ปิดความเสี่ยงข้อมูลด้วยสำรองรายวันและ Copy-then-Swap** ก่อนเพิ่มฟีเจอร์ใด ๆ
เพราะข้อมูล Log คือทรัพย์สินที่สร้างไม่ได้อีกเมื่อหาย
- **3. ติดตั้งช่องทางแจ้ง Error เข้า Telegram ผู้ดูแล** พร้อมแก้ Silent waitLock เพื่อตัดความกำกวมระหว่าง ไม่มีเหตุการณ์กับยังไม่ได้รับ
- **4. เฝ้าไควตา LINE ด้วยการอ่านสถานะไควตาคงเหลือ** และตั้งเกณฑ์แจ้งเตือนล่วงหน้า
เพราะนี่คือข้อจำกัด ที่เคยสร้างปัญหาจริงและจะกลับมาอีกเมื่อฟลิตโต
- **5. ย้ายความลับที่เหลืลง PropertiesService และ Escape พารามิเตอร์ของ WebApp**
เพื่อให้การแฮ็ค และการส่งลิงก์ปลอดภัยขึ้นโดยแทบไม่เปลี่ยนพฤติกรรมระบบ
- **6. วางแผนอายุข้อมูลตั้งแต่วันนี้** ด้วยการแบ่งซีตรายปีและนโยบาย Archive
แม้จะเป็นงานระดับ P2
เพราะข้อมูลวันนี้คือฐานที่ระบบรายงานเดือนจะยืนอยู่บนมันในอีกสองปีข้างหน้า

หากดำเนินการตามลำดับข้างต้น ระบบจะพร้อมก้าวสู่รุ่นถัดไปอย่างมั่นคง ไม่ว่าจะเป็นการเปิดใช้ Looker Studio สำหรับผู้บริหาร การเปิด AppSheet ให้หัวหน้างานใช้ในมือถือ หรือการผูกภาพเหตุการณ์เข้ากับทุกการแจ้งเตือน ซึ่งทั้งหมดรวมกันยกระดับ Fleet Safety Intelligence V2 จากระบบติดตามและแจ้งเตือน ให้เป็นเครื่องมือ ตรวจสอบเหตุการณ์ครบวงจรที่หลักฐานทุกชั้นเชื่อมโยงกันตั้งแต่พิกัด คะแนน จนถึงภาพวิดีโอในวินาทีที่เกิดขึ้นจริง